

# Printing and variables

Revise 360 lesson plan — 60 minutes

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Unit            | Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3                                                     |
| Lesson          | Printing and variables (lesson 1 of the unit)                                                                     |
| Lesson ID       | pr-l01                                                                                                            |
| Experience ID   | pr-l01                                                                                                            |
| Plan ID         | PLAN-PR-L01                                                                                                       |
| Content version | b29934e90454                                                                                                      |
| Website         | <a href="https://revise360.co.uk/experience.html?id=pr-l01">https://revise360.co.uk/experience.html?id=pr-l01</a> |

## Big question, outcomes and success criteria

**Big question** — Why must a program decide the data type of a value before it can do anything useful with it?

### Prerequisites

None within this unit: it is the first lesson. Treat the starter as a baseline check rather than retrieval of something you have taught.

### Assessable outcomes

1. **pr-l01-o1** — know how to print lines of output in the order a task asks for
2. **pr-l01-o2** — know how to hold a value in a variable or a constant, and how to read one with input()
3. **pr-l01-o3** — understand why a value that has been typed in must be cast before it can be calculated with

### Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- The words appear on the right, under Program output. **pr-l01-s1-a1**
- Change only the word inside the quotation marks. **pr-l01-s1-a2**
- Display exactly: Goodbye **pr-l01-s1-a2**
- Replace \_\_\_\_ with one instruction of your own. **pr-l01-s1-a4**
- Display exactly: Line one **pr-l01-s1-a4**
- Nothing is typed in by the user. **pr-l01-s1-a5**
- Write your program under the line starting with # . **pr-l01-s1-a5**
- Display exactly: Hello world **pr-l01-s1-a5**

## Vocabulary, resources and preparation

### Key vocabulary — Variable, Constant, Casting, Concatenation

### Resources

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| SLIDES            | presentations/PR_L01_PrintingAndVariables_Lesson.pptx |
| WORKSHEET         | worksheets/PR_L01_PrintingAndVariables_Worksheet.docx |
| ONLINE EXPERIENCE | experience.html?id=pr-l01                             |

## Before the lesson

1. Sign in on the **teacher dashboard** (teacher.html) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

## How pupils get in

1. Go to topics.html on revise360.co.uk.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

**Worth knowing before you promise anything.** In this build APP\_CONFIG.backendUrl is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

## Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

| MIN   | PHASE                                     | TEACHER                                                                                      | PUPILS                                                       | EVIDENCE TO CHECK                                                                |
|-------|-------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------|
| 0-5   | <b>Retrieve</b>                           | Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites. | Answer the starter on the worksheet.                         | The starter box on PR_L01_PrintingAndVariables_Worksheet.docx is filled in.      |
| 5-8   | <b>Review the starter</b>                 | Take answers; correct the one most got wrong before moving on.                               | Correct their own starter.                                   | A correction written, not just a tick.                                           |
| 8-16  | <b>Explain and model</b>                  | The new idea, then the worked example, typed live rather than shown finished.                | Watch the example being built, and say what each line does.  | They can name the line that does the job, not just copy it.                      |
| 16-19 | <b>Set up and settle</b>                  | Sign-in, launch, and what to record on paper before anyone types.                            | Sign in, open the lesson, read the task before typing.       | Everyone is at station 1 activity 1, not typing yet.                             |
| 19-48 | <b>Online: work the stations in order</b> | Pupils work; you circulate and read the editor over shoulders.                               | Work the stations in order, running and checking as they go. | The station badges fill in; the editor shows their own program, not the starter. |
| 48-53 | <b>Check understanding</b>                | The hinge question, hands down, then the reteach if the class splits.                        | Answer the hinge question, hands down.                       | The split across the options, before any reteach.                                |
| 53-58 | <b>Apply or extend</b>                    | The challenge activity, or the exam practice on the worksheet.                               | The Challenge activity, or the exam practice.                | worksheet, Exam practice a) to c)                                                |

| MIN   | PHASE                                  | TEACHER                                                | PUPILS                 | EVIDENCE TO CHECK                           |
|-------|----------------------------------------|--------------------------------------------------------|------------------------|---------------------------------------------|
| 58-60 | <b>Exit and set up the next lesson</b> | The exit question and what to finish before next time. | Write the exit answer. | One sentence that answers the big question. |

## The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

### 1. Printing output

- `print()` puts one value on the screen and then moves down to the next line.
- Text to be shown exactly as written goes inside quotation marks, and one `print()` gives one line of output.
- Common mistake: leaving the quotation marks off, so Python hunts for a variable with that name instead.

**Challenge on this wall:** Why does Python treat `ready` and `"ready"` as two completely different things?

### 2. Variables

- A variable is a named box in memory holding one value, and the name always goes on the left of the equals sign.
- A constant is a value the program never changes once it is set, and its name is written in capitals.
- Common mistake: a second assignment replaces the value, it does not add to it.

**Challenge on this wall:** What becomes of the first value when the same variable is given a second one?

### 3. Reading input

- `input()` stops the program, waits for a line to be typed and hands that line back.
- Store what comes back in a variable, or the value the user typed is thrown away.
- Common mistake: reading two answers into the same variable, so the first one is lost.

**Challenge on this wall:** Why can a program print its answers in a different order from the one it asked the questions in?

### 4. Numbers and text

- Integers count whole things, floats hold decimals, strings hold text and booleans hold `True` or `False`.
- `+` adds two numbers but joins two strings end to end, so the data type decides what `+` means.
- Common mistake: joining a number to a string with `+`, which Python refuses to do.

**Challenge on this wall:** Which data type would you choose for a value of 1.50, and why?

### 5. Casting

- Casting changes a value from one data type into another.
- `int()` and `float()` turn text into a number, and `str()` turns a number into text.
- Common mistake: adding to what `input()` handed back without casting it first, which joins the digits on the end instead.

**Challenge on this wall:** Why does `int(input())` appear in so many programs?

### 6. Putting it together

- A whole program reads its values, works something out and then prints the lines it was asked for.
- Decide the data type each value needs before you write a single line.
- Common mistake: printing the lines in a different order from the one the task asked for.

**Challenge on this wall:** Which parts of a program have to happen before anything can be printed?

### The example to model

Python is a language for writing instructions. A computer follows them in order, from the top. `print()` is the instruction for putting one line on the screen.

```
print("Hello")
```

It displays **Hello**

**Decisions to model out loud, not just the finished answer:** say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

## Retrieval and the check for understanding

**Starter** — Three lines run in this order: a = 40, then a = 65, then print(a). Write down exactly what is printed, and explain in one sentence what became of the first value.

Answer space: 3 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

### Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-101-s3-c7), so the wording a pupil sees matches the wording you say.

**The user types red and then blue. What will this program display?**

```
first = input()
second = input()
print(second)
print(first)
```

**Correct:** blue red

### What a pupil might choose instead

- red blue
- red red

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

**The course's own response to a wrong answer here:** The reading order and the displaying order are separate. The first line typed goes into first, whatever happens next.

**One response, not four.** one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

### Misconceptions this lesson addresses

- leaving the quotation marks off, so Python hunts for a variable with that name instead. [wall panel](#)
- Python is looking for something named "Ready", because nothing tells it that Ready is text. [pr-101-s1-a6, a Fix it activity](#)
- a second assignment replaces the value, it does not add to it. [wall panel](#)
- Which side of the = does a name go on? [pr-101-s2-a7, a Fix it activity](#)
- reading two answers into the same variable, so the first one is lost. [wall panel](#)
- Is the answer being kept anywhere? Look at what is missing before the input(). [pr-101-s3-a5, a Fix it activity](#)
- joining a number to a string with +, which Python refuses to do. [wall panel](#)
- Read the error. What two kinds of value is Python being asked to join? [pr-101-s4-a6, a Fix it activity](#)

## Guided to independent practice

| STAGE     | ACTIVITIES | WHAT IT IS                                                                                                                |
|-----------|------------|---------------------------------------------------------------------------------------------------------------------------|
| Recognise | 16         | The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.                   |
| Construct | 10         | Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer. |

| STAGE     | ACTIVITIES | WHAT IT IS                                                                                                        |
|-----------|------------|-------------------------------------------------------------------------------------------------------------------|
| Produce   | 24         | The pupil writes it from the brief and nothing else.                                                              |
| Challenge | 6          | The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station. |

### Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

### Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

## Delivering the online experience

### Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

### The stations, and what to record at each

| #     | STATION                             | ACTIVITIES | RECORD ON PAPER                                                                                    |
|-------|-------------------------------------|------------|----------------------------------------------------------------------------------------------------|
| 1     | Printing output                     | 10         | Each print() shows one value and then moves on to...                                               |
| 2     | Variables                           | 10         | In programming = means 'put this in here', so a second assignment does not add to the value, it... |
| 3     | Reading input                       | 10         | input() is patient: a program will sit on that line until...                                       |
| 4     | Numbers and text                    | 9          | Joining two strings end to end has its own name:...                                                |
| 5     | Casting                             | 9          | Cast is borrowed from metalwork: the value is poured into...                                       |
| 6     | Putting it together                 | 7          | Input, process, output is such a common pattern that it is shortened to...                         |
| final | Final challenge: name the data type | 1          | the answer to the station's challenge                                                              |

### Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

## Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

**Python: the order is not negotiable.** Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

## Assessment, evidence and unfinished work

| ID        | QUESTION                                                                                                                                                                                                                 | MARKS | WHERE                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--------------------------------------------------------|
| pr-101-x1 | A pupil writes two lines to add 1 to a number that has been typed in: <code>n = input()</code> and then <code>print(n + 1)</code> . State what Python does when these two lines run, and rewrite them so that they work. | 3     | worksheet, Exam practice a); deck, Exam practice slide |
| pr-101-x2 | Write a program that reads a word and then a whole number, adds 1 to the number, and prints the word on one line and the new number on the next line.                                                                    | 4     | worksheet, Exam practice b); deck, Exam practice slide |
| pr-101-x3 | Write a program that reads a number which may have a decimal point, and prints one line in the form Value is 4.0, with the number shown as a real number.                                                                | 3     | worksheet, Exam practice c); deck, Exam practice slide |

## Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-101.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-101.json`.

## Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — Ready Steady Go; Go Steady Ready
- **What will this program display?** — 8 8; 5 + 3 5 + 3
- **What will this program display?** — red red; colour colour
- **What will this program display?** — cat; catdog
- **The user types Sam. What will this program display?** — Sam; name name

## Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

## Exit, next lesson and homework

### Exit check

One of these, written, not a show of hands:

- Printing lines of output in the right order
- Holding values in variables and constants
- Casting between text and numbers

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

### **Retrieval link to next lesson**

Next is pr-102, Arithmetic. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

### **Homework**

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

## **Shorter lessons, longer lessons and no internet**

---

### **If this lesson takes more than an hour**

It will. There are 56 activities across 6 stations, and a beginner does not finish 56 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

### **A 40 to 45 minute lesson**

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

### **If the internet or the devices are unavailable**

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

---

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.